

Name: - Shubham Das S. A. P I. D: - 590033500
Date: - 13/09/2026 Programme: - B. TECH (CSE)

Assignments: - Prepare an algorithm for each operation and a test-case table covering valid, invalid, and boundary inputs. Then develop, compile, and execute a menu-driven modular C project that calculates the geometrical measurements and physical properties of selected solids. Test the program using the prepared test cases.
[Note: Use only fundamental data types, functions with call by value, selection statements, and loops. Do not use recursion, arrays, pointer variables, pointer parameters, structures, unions, enumerations, *typedef*, or functions from *math.h*. Assume that all input values and calculated results are within the ranges supported by their respective data types.]

Part 1: - Thorough understanding of the problem statement

Input: - Geometrical Dimensions: - *radius, height, length, breadth*
(Variables: *double*, Units: meters).

Physical parameters: *density, fluid_density* (Variables: *double*, Units: kg/m³),
elevation (Variable: *double*, Units: meters).

Menu selections: *choice, solid_choice* (Variables: *int*, Units: unitless)

Output: - Area calculations (Variable: *double*, Units: cubic metres).
Volume calculations (Variable: *double*, Units: cubic meters).
Physics metrics: Mass (kg), Weight (N), Potential Energy (J), Pressure (Pa),
Buoyant Force (N) (Variables: *double*).

Formula: - Sphere area: $4 * PI * radius * radius$;
Sphere Volume: $(4.0/3.0) * PI * radius * radius$;
Cylinder Area: $2 * PI * radius * (radius + height)$;
Cone Slant Height: $calculate_square_root(radius * radius + height * height)$;
Mass: $density * volume$;
Pressure: $(density * volume * GRAVITY) / contact_area$;
Newton-Raphson Square Root:
 $new_value = 0.5 * (old_value + number / old_value)$;
loop while $difference >= TOLERANCE$;

Part 2: - Writing, compiling and evaluating the programme

Step 1: - Start

Step 2: - Declare modular variables across multiple files for main applications flow,

geometric transformations, and physics derivations.

Step 3 (Main Control): - Initiate a *do-while* loop displaying the 10-option main menu. Obtain *choice*. Validate *choice* ≥ 1 and ≤ 10 .

Step 4 (Branching): - Execute a *switch-case* evaluating *choice*;

Case 1-3 (Geometry): Prompt for solid type (Sphere, Cylinder, Cone, Cuboid). Reject Sphere for Base Area calculations. Prompt for respective dimensions (strictly > 0). Call appropriate *geometry.c* function and print the result.

Case 4-9 (*Physics*): Prompt for required top-level physics variables (e.g., fluid density or elevation). Pass these values to the respective *physics.c* function. Inside *physics.c*, prompt for the solid type and dimensions, calculate the intermediate volume/area, compute the physical property, and return the metric to *main.c* for display.

Step 5 (Utility Logic): For any square root requirement (like cone slant height), invoke the custom Newton-Raphson iteration from *math_utils.c* bypassing *<math.h>*.

Step 6: - Loop until user selects 10 (Exit). Stop.

Part 3: - Writing and compiling the C programme

File 1: - *solid.h*

```

GNU nano 9.0
#ifndef SOLID_H
#define SOLID_H

// Constants as per documentation
#define PI 3.1416
#define GRAVITY 9.8
#define TOLERANCE 0.000001

// Solid Identifiers
#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4

// Buoyancy States
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

// math_utils.c
double calculate_square_root(double number);

// geometry.c
double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

// physics.c
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

#endif

```

File 2: - *math_utils.c*

```
GNU nano 9.0
#include "solid.h"

double calculate_square_root(double number) {
    if (number < 0) {
        return -1;
    }
    if (number == 0) {
        return 0;
    }
    double old_value = number;
    double new_value, difference;
    do {
        new_value = 0.5 * (old_value + number / old_value);
        difference = new_value - old_value;
        if (difference < 0) {
            difference = -difference;
        }
        old_value = new_value;
    } while (difference >= TOLERANCE);
    return new_value;
}
```

File 3: - *geometry.c*

```

GNU nano 9.0
#include "solid.h"

double sphere_area(double radius) {
    return 4 * PI * radius * radius;
}

double sphere_volume(double radius) {
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height) {
    return 2 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height) {
    return PI * radius * radius * height;
}

double cone_area(double radius, double height) {
    double slant_height = calculate_square_root(radius * radius + height * height);
    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height) {
    return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height) {
    return 2 * (length * breadth + breadth * height + length * height);
}

double cuboid_volume(double length, double breadth, double height) {
    return length * breadth * height;
}

double circular_base_area(double radius) {
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth) {
    return length * breadth;
}

```

File 4: - *physics.c*

```

GNU nano 9.0
#include <stdio.h>
#include "solid.h"

// Internal helper for physical dimension gathering without violating pointer restrictions
double get_positive_dim(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val <= 0) printf("Error: Dimension must be greater than zero.\n");
    } while (val <= 0);
    return val;
}

double calculate_mass(double density) {
    int type;
    double vol = 0;
    do {
        printf("Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
        scanf("%d", &type);
        if (type < 1 || type > 4) printf("Error: Invalid choice.\n");
    } while (type < 1 || type > 4);

    if (type == SPHERE) {
        double r = get_positive_dim("Enter radius (m): ");
        vol = sphere_volume(r);
    } else if (type == CYLINDER) {
        double r = get_positive_dim("Enter radius (m): ");
        double h = get_positive_dim("Enter height (m): ");
        vol = cylinder_volume(r, h);
    } else if (type == CONE) {
        double r = get_positive_dim("Enter radius (m): ");
        double h = get_positive_dim("Enter height (m): ");
        vol = cone_volume(r, h);
    } else if (type == CUBOID) {
        double l = get_positive_dim("Enter length (m): ");
        double b = get_positive_dim("Enter breadth (m): ");
        double h = get_positive_dim("Enter height (m): ");
        vol = cuboid_volume(l, b, h);
    }

    return density * vol;
}

double calculate_weight(void) {
    double density = get_positive_dim("Enter density of the solid (kg/m^3): ");
    double mass = calculate_mass(density);
}

^G Help      ^O Write Out  ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File  ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line

```

GNU nano 9.0

```
}

double potential_energy(double elevation) {
    double density = get_positive_dim("Enter density of the solid (kg/m^3): ");
    double mass = calculate_mass(density);
    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density) {
    int type;
    double vol = 0, area = 0;
    do {
        printf("Select Solid resting on base (2-Cylinder, 3-Cone, 4-Cuboid): ");
        scanf("%d", &type);
        if (type < 2 || type > 4) printf("Error: Solid must be 2, 3, or 4 (No Spheres).\n");
    } while (type < 2 || type > 4);

    if (type == CYLINDER) {
        double r = get_positive_dim("Enter radius (m): ");
        double h = get_positive_dim("Enter height (m): ");
        area = circular_base_area(r);
        vol = cylinder_volume(r, h);
    } else if (type == CONE) {
        double r = get_positive_dim("Enter radius (m): ");
        double h = get_positive_dim("Enter height (m): ");
        area = circular_base_area(r);
        vol = cone_volume(r, h);
    } else if (type == CUBOID) {
        double l = get_positive_dim("Enter length (m): ");
        double b = get_positive_dim("Enter breadth (m): ");
        double h = get_positive_dim("Enter height (m): ");
        area = cuboid_base_area(l, b);
        vol = cuboid_volume(l, b, h);
    }

    return (density * vol * GRAVITY) / area;
}

double buoyant_force(double fluid_density) {
    double vol = 0;
    int type;
    do {
        printf("Select Submerged Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
        scanf("%d", &type);
        if (type < 1 || type > 4) printf("Error: Invalid choice.\n");
    } while (type < 1 || type > 4);
}
```

 Help
 Exit

 Write Out
 Read File

 Where Is
 Replace

 Cut
 Paste

 Execute
 Justify

 Location
 Go To Line

```

GNU nano 9.0
    area = cuboid_base_area(l, b);
    vol = cuboid_volume(l, b, h);
}
return (density * vol * GRAVITY) / area;
}

double buoyant_force(double fluid_density) {
    double vol = 0;
    int type;
    do {
        printf("Select Submerged Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
        scanf("%d", &type);
        if (type < 1 || type > 4) printf("Error: Invalid choice.\n");
    } while (type < 1 || type > 4);

    if (type == SPHERE) {
        double r = get_positive_dim("Enter radius (m): ");
        vol = sphere_volume(r);
    } else if (type == CYLINDER) {
        double r = get_positive_dim("Enter radius (m): ");
        double h = get_positive_dim("Enter height (m): ");
        vol = cylinder_volume(r, h);
    } else if (type == CONE) {
        double r = get_positive_dim("Enter radius (m): ");
        double h = get_positive_dim("Enter height (m): ");
        vol = cone_volume(r, h);
    } else if (type == CUBOID) {
        double l = get_positive_dim("Enter length (m): ");
        double b = get_positive_dim("Enter breadth (m): ");
        double h = get_positive_dim("Enter height (m): ");
        vol = cuboid_volume(l, b, h);
    }

    return fluid_density * vol * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density) {
    if (solid_density > fluid_density) {
        return SINKS;
    } else if (solid_density < fluid_density) {
        return FLOATS;
    } else {
        return SUSPENDED;
    }
}
}

```

File 5: - main.c

```

GNU nano 9.0
#include <stdio.h>
#include "solid.h"

// Helper for validating numeric inputs strictly greater than zero in main
double validate_positive(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val <= 0) printf("Error: Value must be greater than zero.\n");
    } while (val <= 0);
    return val;
}

int main() {
    int choice, solid;
    double r, h, l, b, ans;

    do {
        printf("\n===== PHYSICS & GEOMETRY CALCULATOR =====\n");
        printf("1. Surface area\n2. Volume\n3. Base Area\n4. Mass\n");
        printf("5. Weight\n6. Gravitational potential energy\n7. Pressure on the base\n");
        printf("8. Buoyant force\n9. Buoyancy behavior\n10. Exit\n");
        printf("=====\n");
        printf("Enter operation (1-10): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
            case 2:
                do {
                    printf("Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
                    scanf("%d", &solid);
                } while (solid < 1 || solid > 4);

                if (solid == SPHERE) {
                    r = validate_positive("Radius (m): ");
                    ans = (choice == 1) ? sphere_area(r) : sphere_volume(r);
                } else if (solid == CYLINDER) {
                    r = validate_positive("Radius (m): ");
                    h = validate_positive("Height (m): ");
                    ans = (choice == 1) ? cylinder_area(r, h) : cylinder_volume(r, h);
                } else if (solid == CONE) {
                    r = validate_positive("Radius (m): ");
                    h = validate_positive("Height (m): ");
                    ans = (choice == 1) ? cone_area(r, h) : cone_volume(r, h);
                } else if (solid == CUBOID) {
                    l = validate_positive("Length (m): ");

```

```
        ans = (choice == 1) ? cone_area(r, h) : cone_volume(r, h);
    } else if (solid == CUBOID) {
        l = validate_positive("Length (m): ");
        b = validate_positive("Breadth (m): ");
        h = validate_positive("Height (m): ");
        ans = (choice == 1) ? cuboid_area(l, b, h) : cuboid_volume(l, b, h);
    }

    if (choice == 1) printf("Surface Area: %.4f sq.m\n", ans);
    else printf("Volume: %.4f cu.m\n", ans);
    break;

case 3:
do {
    printf("Select Solid for Base (2-Cylinder, 3-Cone, 4-Cuboid): ");
    scanf("%d", &solid);
    if (solid == SPHERE) printf("Error: Base area not defined for sphere. Select another.\n");
} while (solid < 2 || solid > 4);

if (solid == CYLINDER || solid == CONE) {
    r = validate_positive("Radius (m): ");
    printf("Base Area: %.4f sq.m\n", circular_base_area(r));
} else {
    l = validate_positive("Length (m): ");
    b = validate_positive("Breadth (m): ");
    printf("Base Area: %.4f sq.m\n", cuboid_base_area(l, b));
}
break;

case 4:
    ans = validate_positive("Enter density (kg/m^3): ");
    printf("Mass: %.4f kg\n", calculate_mass(ans));
    break;

case 5:
    printf("Weight: %.4f N\n", calculate_weight());
    break;

case 6:
    ans = validate_positive("Enter elevation (m): ");
    printf("Potential Energy: %.4f J\n", potential_energy(ans));
    break;

case 7:
    ans = validate_positive("Enter density (kg/m^3): ");
    printf("Pressure on base: %.4f Pa\n", calculate_pressure(ans));
    break;
```

```
GNU nano 9.0

    case 4:
        ans = validate_positive("Enter density (kg/m^3): ");
        printf("Mass: %.4f kg\n", calculate_mass(ans));
        break;

    case 5:
        printf("Weight: %.4f N\n", calculate_weight());
        break;

    case 6:
        ans = validate_positive("Enter elevation (m): ");
        printf("Potential Energy: %.4f J\n", potential_energy(ans));
        break;

    case 7:
        ans = validate_positive("Enter density (kg/m^3): ");
        printf("Pressure on base: %.4f Pa\n", calculate_pressure(ans));
        break;

    case 8:
        ans = validate_positive("Enter fluid density (kg/m^3): ");
        printf("Buoyant Force: %.4f N\n", buoyant_force(ans));
        break;

    case 9:
        {
            double sd = validate_positive("Enter solid density (kg/m^3): ");
            double fd = validate_positive("Enter fluid density (kg/m^3): ");
            int state = determine_buoyancy(sd, fd);
            if (state == SINKS) printf("Behavior: Solid Sinks.\n");
            else if (state == FLOATS) printf("Behavior: Solid Floats.\n");
            else printf("Behavior: Solid is Suspended.\n");
        }
        break;

    case 10:
        printf("Exiting... Goodbye!\n");
        break;

    default:
        printf("Error: Invalid choice.\n");
}
} while (choice != 10);

return 0;
}
```

Part 4: - Running and executing the algorithm on a platform (Kali Linux VM)

- Step 1: - Open the terminal
- Step 2: - Create the source code
- Step 3: - Write and Save Code
- Step 4: - Compile the programme

Step 5: - Run the Binary Executable Programme

```
(kali㉿kali)-[~]  
$ nano solid.h  
  
(kali㉿kali)-[~]  
$ nano math_utils.c  
  
(kali㉿kali)-[~]  
$ nano geometry.c  
  
(kali㉿kali)-[~]  
$ nano physics.c  
  
(kali㉿kali)-[~]  
$ nano main.c  
  
(kali㉿kali)-[~]  
$ gcc main.c geometry.c physics.c math_utils.c -o project  
  
(kali㉿kali)-[~]  
$ ./project
```

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): 1

Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 1

Radius (m): 2.0

Surface Area: 50.2656 sq.m

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): 2

```
Enter operation (1-10): 2
Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 2
Radius (m): 3.0
Height (m): 4.0
Volume: 113.0976 cu.m
```

```
===== PHYSICS & GEOMETRY CALCULATOR =====
```

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

```
=====
Enter operation (1-10): 9
Enter solid density (kg/m^3): 800
Enter fluid density (kg/m^3): 1000
Behavior: Solid Floats.
```

```
===== PHYSICS & GEOMETRY CALCULATOR =====
```

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

```
=====
Enter operation (1-10): 4
Enter density (kg/m^3): -5
Error: Value must be greater than zero.
Enter density (kg/m^3): 50
Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 3
Enter radius (m): 10
Enter height (m): 8
Mass: 41888.0000 kg
```

```
===== PHYSICS & GEOMETRY CALCULATOR =====
```

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base

7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

```
=====
Enter operation (1-10): 4
Enter density (kg/m^3): -5
Error: Value must be greater than zero.
Enter density (kg/m^3): 50
Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 3
Enter radius (m): 10
Enter height (m): 8
Mass: 41888.0000 kg
```

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

```
=====
Enter operation (1-10): 5
Enter density of the solid (kg/m^3): 500
Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 1
Enter radius (m): 50
Weight: 2565640000.0000 N
```

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

```
=====
Enter operation (1-10): 3
Select Solid for Base (2-Cylinder, 3-Cone, 4-Cuboid): 1
Error: Base area not defined for sphere. Select another.
Select Solid for Base (2-Cylinder, 3-Cone, 4-Cuboid): 2
Radius (m): 8
Base Area: 201.0624 sq.m
```

===== PHYSICS & GEOMETRY CALCULATOR =====

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): 3

Select Solid for Base (2-Cylinder, 3-Cone, 4-Cuboid): 3

Radius (m): 0.001

Base Area: 0.0000 sq.m

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): 4

Enter density (kg/m³): 10

Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 3

Enter radius (m): 0.001

Enter height (m): 0

Error: Dimension must be greater than zero.

Enter height (m): 10

Mass: 0.0001 kg

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): █

3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): 4

Enter density (kg/m³): 10

Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 3

Enter radius (m): 0.001

Enter height (m): 0

Error: Dimension must be greater than zero.

Enter height (m): 10

Mass: 0.0001 kg

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): 1

Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): 3

Radius (m): 3

Height (m): 4

Surface Area: 75.3984 sq.m

===== PHYSICS & GEOMETRY CALCULATOR =====

1. Surface area
2. Volume
3. Base Area
4. Mass
5. Weight
6. Gravitational potential energy
7. Pressure on the base
8. Buoyant force
9. Buoyancy behavior
10. Exit

=====

Enter operation (1-10): 10

Exiting... Goodbye!

└─(kali@kali) - [~]

Step 5: - Standard Test Cable

TEST CASE	SCENARIO OPERATION	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
1	Surface Area (Sphere)	choice = 1 solid = 2.0	Surface Area: 50.2656 sq.m	Surface Area: 50.2656 sq.m	PASS
2	Volume (Cylinder)	choice = 2 solid = 2 radius = 3.0 height = 4.0	Volume: 113.0976 cu.m	Volume: 113.0976 cu.m	PASS
3	Buoyancy Behaviour	choice = 9 solid_density = 800 fluid_density = 1000	Behavior: Solid Floats.	Behavior: Solid Floats.	PASS
4	Invalid Input Validation (Density)	choice = 4 density = -5	Error: Value must be greater than zero	Error: Value must be greater than zero	PASS
5	Mass Calculation (Sphere)	choice = 4 density = 50 solid = 3 radius = 10 height = 8	Mass: 41888.0000 kg	Mass: 41888.0000 kg	PASS
6	Base Area (Cylinder)	choice = 3 solid = 2 radius = 0.001	Weight: 2565640000.0000 N	Weight: 2565640000.0000 N	PASS
7	Invalid Selection Validation (Sphere Base Area)	choice = 3 solid = 1	Error: Base area not defined for sphere. Select another.	Error: Base area not defined for sphere. Select another.	PASS
8	Base Area (Cylinder)	choice = 3 solid = 2 radius = 8	Base Area: 201. 0624 sq.m	Base Area: 201.0624 sq.m	PASS
9	Base Area Precision (Cone)	choice = 3 solid = 3	Base Area: 0.0000 sq.m	Base Area: 0.0000 sq.m	PASS

		radius = 0.001			
10	Invalid Dimension Validation (Height Zone)	choice = 4 density = 10 solid = 3 radius = 0.001 height = 10	Error: Dimension must be greater than zero.	Error: Dimension must be greater than zero.	PASS
11	Mass Small Value Calculation (Cone)	choice = 4 density = 10 solid = 3 radius = 0.001 height = 10	Mass: 0.0001 kg	Mass: 0.0001 kg	PASS
12	Surface Area (Cone-Newton-Raphson Sqrt Test)	choice = 1 solid = 3 radius = 3 height = 4	Surface Area: 75.3984 sq.m	Surface Area: 75.3984 sq.m	PASS
13	Exit Program	choice = 10	Exiting... Goodbye!	Exiting... Goodbye!	PASS

Step 6: - Evaluation of the algorithm

The input ingestion cycle is completely fault-tolerant; *do-while* constructs actively intercept out-of-domain topological requests (such as querying a base area for a Sphere) locally inside the menu state machines prior to function calls, preventing arithmetic breakdown.